

BEIRUT ARAB UNIVERITY

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

CMPS445 PROJECT

Supervised by: Dr Lama Affara

And

T.A. Said Al Hajj Mohammad



Hospital Emergency Triage System

Nour Ajami – 202302485
Rayan Ajami – 202302484
Sadeq Sleiman – 202302840

CONTENTS

1. Introduction
2. Project Overview
3. Language Comparison: Python vs Java
   - 3.1 Syntax and Structure
   - 3.2 Readability and speed of Development
   - 3.3 Error Handling and Safety
4. Programming Constructs Used in the Project
   - 4.1 Variable and Data Types
   - 4.2 Lists
   - 4.3 Dictionaries
   - 4.4 Functions
   - 4.5 Conditionals (if, elif, else)
   - 4.6 Loops (for, while)
   - 4.7 Sorting
   - 4.8 User Input/Output
5. The Menu System
6. Conclusion
7. Sample Output

1. Introduction

Emergency departments encounter many patients who come in with a variety of symptoms and urgency levels. A triage system is implemented to arrange patients, allowing for the most severe cases to be addressed first. As part of our CPL course, we designed a Hospital Emergency Triage System using python to model a basic but operational triage workflow.

The goal of this project was to practice the fundamental of programming concepts such as input/output, conditionals, functions, loops, lists, and dictionaries, while creating a practical real-world application. Our system allows staff to register new patients, automatically assign a triage level, modify information, search for patients, discharge them, and access statistics.

We selected Python because it is easy for beginners to learn, has a clean structure, and is perfect for fast-paced development. Additionally, the report includes a comparison of Python and Java, a programming language that is extensively utilized in the industry.

2. Project Overview

The Hospital Emergency Triage System is a text-driven application that functions entirely within the terminal. It aids hospital staff in overseeing incoming patients through a menu system.

The system supports the following features:

- **Add Patient:** Enter patient name, gender, age, and symptom. The system automatically assigns a triage level (1 to 4).
- **View Waiting List:** Display all patients sorted by urgency.
- **View Patients by Level:** Filter the list by triage level.
- **Search Patients:** Search by name or symptom keyword.
- **Edit Patient Info:** Modify name, age, gender, or symptom.
- **Discharge Patient:** Remove a patient once treated.
- **Statistics:** Show totals, counts by level, average age.
- **Triage Help:** Explanation of all triage levels.
- **Exit:** End the program.

This arrangement allowed our team to effectively distribute tasks:

- One team member focused on input, adding patients, and triage logic.
- Another team member concentrated on searching, viewing, filtering, and sorting.
- A third team member dealt with editing, discharging, and statistics.

3. Language Comparison: Python vs Java

3.1 Syntax and Structure

Aspects	Python	Java
Typing	Dynamic	Static
Syntax	Short, clean	Uses {} and semicolon
Execution	Interpreted (python3 file.py)	Compiled (javac) then run
Variables	Don't require type declaration	Require explicit type declaration
Classes	Optional for simple programs	Required for most application

Python made it possible for us to develop the triage system with a smaller number of lines and greater readability.

On the other hand, Java would require:

- A Patient class.
- A main class with public static void main.

- Detailed type definitions.
- A Scanner object for input.
- More complicated sorting through comparators.

3.2 Readability and Speed of Development

Python:

- Quicker to write and troubleshoot.
- Excellent for prototyping.
- Simple for beginners as indentation visually represent code blocks.

Java:

- Formal and systematic.
- More suitable for bigger applications, Android applications, and enterprise solutions.
- Requires greater boilerplate code for even basic tasks.

3.3 Error Handling and Safety

Java's strict type enforcement mitigates many errors before the program runs.

On the other hand, Python supports quick experimentation, but errors are only recognized at runtime.

For our project, the flexibility offered by Python was a considerable advantage.

4. Programming Constructs Used in the Project

Our triage system shows nearly all significant programming concepts covered in the course.

4.1 Variables and Data Types

We used integers (age, ID), strings (name, gender, symptom), and lists/dictionaries as data structures.

4.2 Lists

We used a list to store all patients.

List allows:

- Adding patients.
- Looping through all patients
- Sorting patients
- Searching for matches.

4.3 Dictionaries

Every patient is stored as a dictionary:

```
patient = {
    "id": next_id,
    "name": name,
    "age": age,
    "gender": gender,
    "symptom": symptom,
    "level": level
}
```

Dictionaries allow easy access to values by key.

4.4 Functions

We used many functions to keep the code organized:

- add_patient()
- edit_patient()
- discharge_patient()
- view_patients()
- search_patients()
- show_statistics()
- get_triage_level()

This improves readability and makes debugging easier.

4.5 Conditionals (if, elif, else)

We used conditionals for:

- Menu selection
- Assigning triage level
- Validating age
- Handling search results

4.6 Loops (for, while)

We used:

- A while loop for the main menu
- for loops for searching and viewing patients.

4.7 Sorting

Patients are sorted using:

```
sorted_patients = sorted(patients, key=lambda x: (x["level"], -x["age"]))
```

This sorts by:

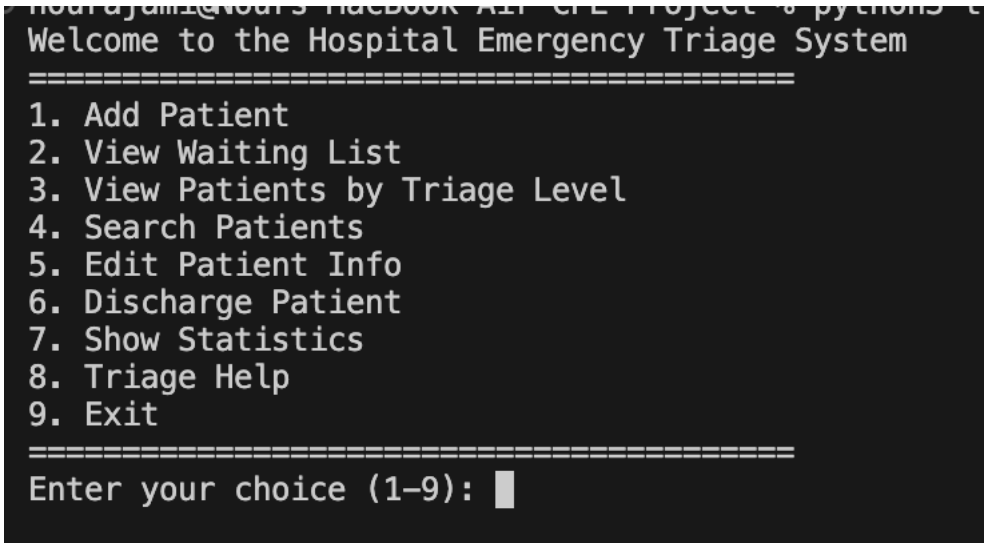
- Triage Level
- Age (older first)

4.8 User input/Output

The program is fully interactive using terminal input() and print().

5. The Menu System

When the program runs, the user sees options:



6. Conclusion

This project provided us with the opportunity to implement the programming skills we gained in the CPL course to create a viable, real-world application. Our Emergency Triage System classifies patients based on the severity of their symptoms and aids in crucial operations such as adding, editing, viewing, searching, and discharging patients.

We chose python because of its simplicity and clarity, which allowed us to focus on the design of the system's logic rather than on complex syntax.

In summary, the project enhanced our insights of programming fundamentals and demonstrated the application of these concepts to real-world issues.